

Aula 04 - Atividade

Descrição: As atividades abaixo adotam a metodologia de **Engenharia Reversa / Rastreamento Mental**

Passo a Passo e Auditoria Crítica de Código com Justificativa de Arquitetura, formatos que exigem defesa técnica pessoal, cálculo manual de registradores e confronto contra as respostas superficiais de modelos de linguagem.

Atividade 1 — Teste de Mesa e Auditoria de Registradores (Análise de Efeitos Colaterais)

Contexto:

Um estagiário escreveu o trecho abaixo para calcular amostras de sensores em um microcontrolador ATmega328, mas o sistema está apresentando leituras corrompidas e travamentos intermitentes de memória.

```
#define SENSOR_PIN 0; // Pino analógico A0  
int pinoLed = 13;    // Pino de sinalização
```

```
void loop() {  
    int x = 10;  
    int y = 4;
```

```
    // Trecho A: Operações e Precedência
```

```
    int a = x++;  
    int b = ++x;  
    x *= y - 2;
```

```
    // Trecho B: Cálculo de Média de Amostragem  
    float mediaAmostras = (x + y) / 2;
```

```
    // Trecho C: Escopo  
    static int contagemCiclos = 0;  
    contagemCiclos++;
```

```
}
```

Comandos:

- Rastreamento de Variáveis:** Sem recorrer a compiladores ou prompts automáticos, execute o teste de mesa do **Trecho A**. Determine e justifique, passo a passo:
 - O valor final contido na variável **a**.
 - O valor final contido na variável **b**.
 - O valor final contido em **x** após a execução da linha **x *= y - 2;** (Atenção à regra de precedência oculta da atribuição composta).
- Diagnóstico da Divisão:** Analise o **Trecho B**. Qual valor numérico exato será armazenado em **mediaAmostras**? Explique por que o resultado difere do esperado matematicamente e proponha a correção exata no código com base na regra de divisão inteira do C.

3. **Identificação de Erros Críticos:** Aponte o erro de compilação introduzido na diretiva `#define` e explique o mecanismo de substituição textual que o causa.

Atividade 2 — Auditoria de Eficiência e Restrição de SRAM (Defesa Técnica)

Contexto:

O microcontrolador ATmega328 possui estritamente **2 kB (2.048 bytes) de SRAM** compartilhados entre variáveis dinâmicas, pilhas de funções e dados de comunicação. O código abaixo precisa ser auditado e reestruturado para cumprir as diretrizes de código limpo e economia de recursos.

```
int ledAlerta = 8;
int statusSistema = 1;
int tempoAtraso = 500;
int leiturasPotenciometro[100]; // Armazenará valores de 0 a 100

void setup() {
    // Configurações
}

void loop() {
    // Loop de controle
}
```

Comandos:

1. Cálculo de Desperdício na SRAM:

- Calcule quantos bytes de SRAM as variáveis globais declaradas acima estão consumindo na arquitetura AVR (8 bits).
- Refaça a tipagem de cada variável aplicando as regras de ouro da aula (escolha do menor tipo primitivo suficiente, modificadores de sinal `unsigned` e a diretiva `#define` ou `const` onde couber).
- Demonstre o novo consumo total em bytes e a porcentagem exata de economia de memória obtida.

2. Defesa Oral / Justificativa Semântica:

- Se o array `leiturasPotenciometro` fosse alterado de `int` para `char` com valores variando entre 0 e 100, haveria risco de estouro (*overflow*)? O que aconteceria se o sensor lesse o valor 200 usando `char` com sinal vs. `unsigned char`?
- Justifique por que declarar pinos físicos como `int` (2 bytes) em vez de `#define` ou `const byte` é considerado uma falha grave de projeto em sistemas com memória restrita.

Entrega:

O resultado de sua atividade deve ser enviado em um único arquivo, no AVA em formato de sua escolha (pdf, docx, txt...).